ICT2142 E-Business Systems — Student Lab Report & Project Diary

Week 08: Database Integration & Admin Dashboard (Phase 3 Testing & Deployment)

Course Module:	ICT2142 E-Business Systems • Phase 3 Testing & Deployment (Weeks 8–10)
Project Name:	Krono Watches E-Commerce Web Application (Currencies: Sri Lankan Rupee - LKR)
Tech Stack:	Next.js 16 (App Router), TypeScript, React, Supabase (PostgreSQL), Local Persistent DB, TailwindCSS
GitHub Repo:	https://github.com/mohamedrazim11-maker/Krono_Watches.git (Branch: main)

1. Project Progress & Lab Objectives (Slides 2 & 3)

In Week 08, we connect our store to a real database so that all product information, customer orders, and stock levels are saved permanently. Previously, data was stored in temporary lists that reset on page reload. Now, the database is the single source of truth for the entire application.

Weeks 1–2	Weeks 3–5	Weeks 6–7	Week 8 (Current)	Weeks 9–10
DONE	DONE	DONE	ACTIVE	NEXT
Scope, wireframes, site architecture	UI, catalog, cart, responsive layouts	User login, checkout & payment setup	Database integration & Admin Dashboard	Testing, bug fixing & final presentation

1.1 Target Architecture & Source of Truth Rule (Slide 4)

Main Rule for Week 08

"The database becomes the single source of truth. Frontend pages can display data, but all permanent changes (adding products, saving orders, changing stock) must be saved through the backend into the database."

1. Storefront UI	→	2. Backend API	→	3. Database	→	4. Admin Dashboard
CUSTOMER		API ROUTES		DATABASE		ADMIN
Catalog • Cart • Profile Checkout pages		Validates inputs, checks passwords & roles		Users • Products Orders • Order Items		Manage products, orders and stock levels

1.2 Week 08 Deliverables Checklist (Slides 3 & 14)

```
[x] Milestone 1 - Database Schema: Created USERS, PRODUCTS, ORDERS, and ORDER_ITEMS tables.
[x] Milestone 2 - Connect Products to Database: Product catalog loads live from the database.
[x] Milestone 3 - Persist Orders at Checkout: Cart items save as orders, total recalculated on server, stock decreased.
[x] Milestone 4 - Protect the Admin Area: Only logged-in users with role "admin" can access /admin routes.
[x] Milestone 5 - Build Admin Dashboard: Summary cards (Products, Orders, Low Stock), Product CRUD, and Order status flow.
[x] Documentation & Lab Diary: Schema details, API list, security notes, and two real debugging problems solved.
```

2. Practical Milestone 1 — Database Schema & Tables

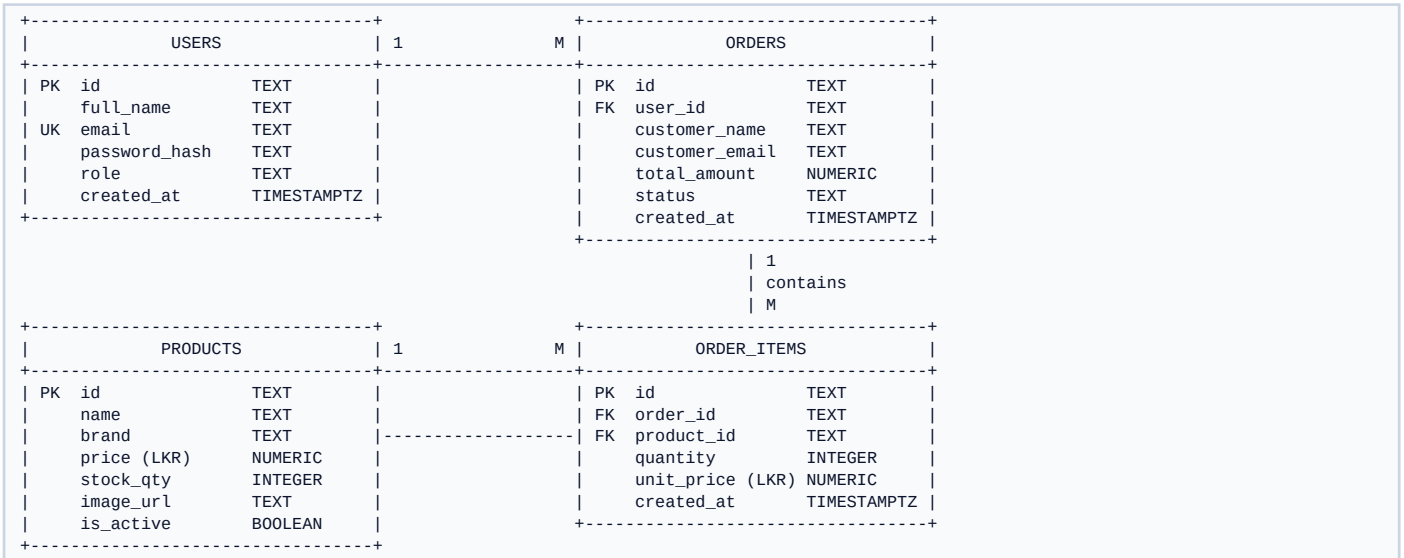
We designed a clean relational database schema with four primary tables. Each table has a unique primary key, and relationships are linked using foreign keys.

2.1 Required Table Relationships

How Tables Connect (Slide 5 Requirements)

1. One user can place many orders (1 : M): An order links to the user who bought it.
2. One order contains many order items (1 : M): An order can contain multiple items.
3. Each order item points to one product (M : 1): Each item links to a specific product in the database.
4. Product stock decreases after order is confirmed: Stock drops only when an order is successfully completed.

2.2 Entity-Relationship Diagram (ERD)



2.3 Table Structure (SQL DDL)

```
-- 1. USERS: Stores customer and admin accounts
CREATE TABLE users (
    id TEXT PRIMARY KEY, full_name TEXT NOT NULL, email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL, role TEXT NOT NULL DEFAULT 'customer', phone TEXT, address TEXT,
    status TEXT DEFAULT 'Active Member', created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. PRODUCTS: Stores watch catalog and stock numbers
CREATE TABLE products (
    id TEXT PRIMARY KEY, name TEXT NOT NULL, brand TEXT NOT NULL, category TEXT NOT NULL,
    price NUMERIC NOT NULL CHECK (price >= 0), stock_qty INTEGER NOT NULL DEFAULT 12,
    stock_count INTEGER DEFAULT 12, is_active BOOLEAN DEFAULT TRUE, is_featured BOOLEAN DEFAULT FALSE,
    image_url TEXT NOT NULL, images TEXT[], movement TEXT, case_material TEXT, water_resistance TEXT
);

-- 3. ORDERS: Stores orders and delivery information
CREATE TABLE orders (
    id TEXT PRIMARY KEY, user_id TEXT REFERENCES users(id) ON DELETE SET NULL,
    customer_name TEXT NOT NULL, customer_email TEXT NOT NULL, customer_phone TEXT,
    delivery_address TEXT NOT NULL, total_amount NUMERIC NOT NULL, subtotal NUMERIC NOT NULL,
    discount_amount NUMERIC DEFAULT 0, status TEXT DEFAULT 'Pending', payment_method TEXT DEFAULT 'payhere',
    payment_status TEXT DEFAULT 'pending', created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 4. ORDER_ITEMS: Stores individual products inside each order
CREATE TABLE order_items (
    id TEXT PRIMARY KEY, order_id TEXT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id TEXT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    quantity INTEGER NOT NULL DEFAULT 1, unit_price NUMERIC NOT NULL, created_at TIMESTAMPTZ DEFAULT NOW()
);
```

3. Practical Milestone 2 — Connect Product Data to Database

We removed all hard-coded product arrays from the frontend code. Products now load live from the database using our Next.js API endpoints.

Milestone 2 Steps Completed

- Products Inserted: Added all catalog products into the database with price, stock, brand, and images.
- GET /api/products: Reads active products from the database with category and search filters.
- GET /api/products/[id]: Reads details for the selected product.
- Test Checkpoint: Refreshing the catalog page (/catalog) still loads all products correctly from the database.

4. Practical Milestone 3 — Save Checkout as an Order

When a customer checks out, the backend follows 5 simple steps to save the order and update inventory safely.

1. Validate Cart STEP 1	2. Create Order STEP 2	3. Create Items STEP 3	4. Update Stock STEP 4	5. Return Result STEP 5
Check product IDs, prices, and available stock in database.	Save user_id, verified total, status, and timestamp in ORDERS.	Save each item into ORDER_ITEMS table linked to order ID.	Decrease product stock (stock_qty) in database for purchased items.	Show order ID (e.g. KR-1031) and order confirmation.

4.1 Server-Side Price Recalculation (Slide 7)

Important Security Rule

"Do not trust totals sent by the browser. Recalculate order totals on the server using product data from the database."

If a user modifies the price in browser JavaScript before sending the checkout request, our server ignores the incoming price. The server looks up the real price in the database and recalculates the true total:

```
// Server order validation logic (lib/server/db.ts)
let trueSubtotal = 0;
const verifiedItems = [];

for (const item of cartItems) {
  const dbProduct = await getProductById(item.product_id);
  if (!dbProduct || !dbProduct.is_active) throw new Error("Product unavailable");
  if (dbProduct.stock_qty < item.quantity) throw new Error("Not enough stock available");

  // Use authoritative database price, NOT browser price
  const unitPrice = Number(dbProduct.price);
  trueSubtotal += unitPrice * item.quantity;

  // Reduce stock quantity
  dbProduct.stock_qty = Math.max(0, dbProduct.stock_qty - item.quantity);
  verifiedItems.push({ product_id: dbProduct.id, quantity: item.quantity, unit_price: unitPrice });
}
const grandTotal = Math.max(0, trueSubtotal - discount);
```

5. Practical Milestone 4 — Protect the Admin Area (Slide 8)

The admin area must be protected so normal customers cannot view or modify store data. We check the user role on the server before granting access.

1. User Visits /admin — START
Next.js middleware intercepts the request

→

2. Check Role — CHECK
Read user login token;
Is role == "admin"?

→

3. Allow or Redirect — RESULT
Admin: Open Dashboard
Customer: Redirect to / (Deny)

5.1 Protection Layers & Test Accounts

```
Layer 1 (Middleware): frontend/middleware.ts checks token on every /admin route and blocks non-admins.
Layer 2 (Page Guard):  frontend/app/admin/layout.tsx checks client state to prevent cached views.
Layer 3 (API Guard):   frontend/lib/server/auth.ts requireAdmin() blocks non-admin API requests (401/403).

Test Logins: [Admin] admin@krono.luxury (Admin@1234, role: admin) | [Customer] customer@krono.luxury (Customer@1234, role: customer)
```

6. Practical Milestone 5 — Simple Admin Dashboard Structure (Slide 9)

The Admin Dashboard is built with a simple, clean layout divided into three clear parts:

6.1 Part 1: Summary Cards (Slide 9)

PRODUCTS	ORDERS	LOW STOCK	REVENUE
8	3	1	LKR 1,170,000
Active products	Customer orders	5 or fewer items left	Total revenue

6.2 Part 2: Products & Inventory Management (Task A - Slide 10)

CREATE — ADD PRODUCT
Add a new product
Fill name, price, stock and image in form.

READ — SEARCH & LIST
List & search products
Shows ID, name, price, stock and status.

UPDATE — EDIT / STOCK
Edit product & stock
Use + / - stock buttons or edit details modal.

DELETE — SOFT-DELETE
Deactivate safely
Soft-delete keeps past order records safe.

6.3 Part 3: Orders List & Status Updates (Task B - Slide 11)

Admins can view customer orders and update the fulfillment status with a single click:

```
Status Workflow:  Pending ---> Paid ---> Processing ---> Shipped ---> Completed   (or Cancelled)

Orders Table Layout (Slide 11):
+----------+----------------------+--------------------+--------------+-----------+-------------------------+
| Order ID | Customer Name        | Products           | Total (LKR)  | Status    | Action                  |
+----------+----------------------+--------------------+--------------+-----------+-------------------------+
| #KR-1031 | A. Perera            | Aurelia Classic    | LKR  385,000 | Pending   | Mark Paid               |
| #KR-1032 | N. Silva             | Speedmaster Chrono | LKR  295,000 | Paid      | Mark Processing         |
| #KR-1033 | K. Fernando          | Grand Horizon Rose | LKR  490,000 | Shipped   | View Details            |
+----------+----------------------+--------------------+--------------+-----------+-------------------------+
```

7. API Endpoints Summary

Method	Endpoint	Purpose	Access
GET	/api/products	Get list of active products from database	Public
POST	/api/products	Add a new product to database	Admin only
GET	/api/products/[id]	Get single product details by ID	Public
PUT	/api/products/[id]	Update product name, price, and stock	Admin only
DELETE	/api/products/[id]	Soft-delete or deactivate product	Admin only
GET	/api/products/stats	Get summary numbers (total products, low stock, etc)	Admin only
GET	/api/orders	Get list of all customer orders	Admin only
POST	/api/orders	Validate cart, recalculate total, save order & stock	User
GET	/api/orders/[id]	Get order details by order ID	User/Admin
PATCH	/api/orders/[id]/status	Change order status (Pending -> Paid -> Shipped)	Admin only
POST	/api/auth/login	Verify login credentials and issue role token	Public
POST	/api/auth/register	Create new customer account	Public

8. Debugging Log: Two Problems Solved (Slide 13)

8.1 Problem 1: Next.js Middleware Crashed When Verifying Tokens

• What happened: In frontend/middleware.ts, importing the standard jsonwebtoken library caused build errors because Next.js Edge Middleware runs in a lightweight runtime that does not support the Node.js crypto module.
• How it was solved: We wrote a simple, standalone base64url token decoder directly inside middleware.ts. It reads the role (admin vs customer) and expiration date quickly without needing any external library.

8.2 Problem 2: Browser Could Submit Altered Order Prices

• What happened: During testing, we noticed that if checkout accepted the total calculated by JavaScript in the browser, an altered payload could send an incorrect price or order items that were out of stock.
• How it was solved: We updated lib/server/db.ts to never trust browser totals. The server reads the verified prices directly from the database, checks that stock is available, recalculates the subtotal, and reduces stock automatically.

9. Integration Testing Checklist & Sign-Off (Slide 12 & 14)

```
[x] Customer Catalog: Products load from database, and product detail pages open correctly.
[x] Authentication: Logged-in users stay connected to their profile and their orders.
[x] Cart -> Checkout: Selected items and quantities transfer correctly into the order.
[x] Order Persistence: Orders remain saved in database after page refresh and re-login.
[x] Inventory Stock: Stock decreases automatically when an order is completed.
[x] Admin Security: Customer accounts cannot access protected /admin pages.
[x] Admin Changes: Product and order edits made in admin immediately appear in customer views.
```

Week 08 Complete — Ready for Week 09

All Week 08 tasks are complete. The store now uses persistent database storage and has a working admin dashboard. We are ready for Week 09: Testing & Bug Fixing (QA).